

Hexapod Brain Roadmap

Whiteboard Strategy Integrated with Current Firmware

E90 Hexapod Team

April 6, 2026

1 Purpose

This document turns the professor meeting whiteboard plan into a concrete implementation roadmap grounded in the current driving code:

- `adaptive_onddevice_hexapod/firmware/hexapod_walking_v2/hexapod_walking_v2.ino`

The goal is to evolve from a deterministic tripod gait controller into an adaptive “brain” that remains safe on hardware while progressively introducing reflexes, online adaptation, and learning-based policy support.

2 Current System Baseline (What Already Exists)

The current firmware already provides a strong and safe base:

1. 12-DOF servo actuation with calibration

- Explicit channel map for all 6 legs and 2 joints per leg (abduction/flexion)
- Per-servo direction and center calibration

2. Reliable actuator interface

- Pololu Maestro serial protocol
- Hard pulse clamps (MIN_US, MAX_US)

3. Deterministic tripod gait state machine

- Tripod A: lift $\rightarrow$ swing+push $\rightarrow$ plant
- Tripod B: lift $\rightarrow$ swing+push $\rightarrow$ plant

4. Smooth interpolation and phase timing

- `moveSmoothTo(...)` with independent step counts and delays
- Existing tunable gait constants (stride amplitudes, flex targets, phase durations)

Interpretation: the robot already has a practical “brain v0.” The roadmap should preserve this safety backbone and layer adaptation on top rather than replacing it.

3 Whiteboard Ideas Mapped to Firmware Reality

Whiteboard concept	Concrete mapping in current stack
TA/TB clock phases	Existing Tripod A/Tripod B phase sequence in firmware loop
P0/P1/P2/P3 phase points	Keyframe targets built through <code>buildPose(...)</code> per sub-phase
Waveform-based generation	Optional future gait generator mode; keep keyframe mode as default safety path
PPO policy block	Learn bounded updates to gait parameters (not raw servo microseconds)
EOM/predictive block	Short-horizon stability predictor from IMU + gait phase for shaping/safety
Terrain/speed conditioning	Add condition vector (terrain class + speed target) to select parameter presets or policy outputs

Table 1: Converting high-level meeting concepts into implementation-ready modules

4 Target Architecture

To stay robust on hardware, architecture is layered and conservative:

Layer 1: Execution (existing)

- Servo command generation, clamping, and Maestro output
- Interpolated motion between pose targets

Layer 2: Parameterized gait generator (next)

- Convert fixed constants into runtime state
- Preserve tripod keyframes as canonical gait

Layer 3: Fast reflex loop (next)

- IMU-based roll/pitch and angular-rate stabilization
- Applied every interpolation tick

Layer 4: Slow adaptation loop (next)

- Once per gait cycle/window, update baseline parameters
- Enforce strict safety bounds and slew-rate limits

Layer 5: Learning policy (later)

- PPO trained in simulation with domain randomization
- Output high-level bounded parameters only

Layer 6: Predictor (optional/late)

- Estimate short-horizon instability risk
- Use for reward shaping and cautious-mode trigger

5 Leg Coupling Specification (Concrete)

Leg indexing and tripod groups

Define leg set:

$$\mathcal{L} = \{\text{FL}, \text{FR}, \text{ML}, \text{MR}, \text{RL}, \text{RR}\}$$

Tripod grouping (matching current firmware):

$$\mathcal{A} = \{\text{FL}, \text{MR}, \text{RL}\}, \quad \mathcal{B} = \{\text{FR}, \text{ML}, \text{RR}\}$$

Joint channels per leg:

$$q_l = (q_l^{\text{abd}}, q_l^{\text{flx}})$$

Phase-coupled clock

Use normalized gait clock $\varphi \in [0, 1)$ over one full stride. Tripod phase offsets:

$$\delta_l = \begin{cases} 0.0, & l \in \mathcal{A} \\ 0.5, & l \in \mathcal{B} \end{cases}$$

and per-leg local phase:

$$\varphi_l = (\varphi + \delta_l) \bmod 1$$

This enforces anti-phase coupling between tripods.

Sub-phase schedule per leg

Each leg has three sub-phases using local phase φ_l :

- Lift: $\varphi_l \in [0, \tau_1)$
- Swing: $\varphi_l \in [\tau_1, \tau_2)$
- Plant/Push: $\varphi_l \in [\tau_2, 1)$

Initial values from v2 timing ratios:

$$\tau_1 \approx \frac{\text{LIFT_STEPS}}{\text{LIFT_STEPS} + \text{SWING_STEPS} + \text{PLANT_STEPS} + \text{PUSH_STEPS}},$$
$$\tau_2 \approx \frac{\text{LIFT_STEPS} + \text{SWING_STEPS}}{\text{LIFT_STEPS} + \text{SWING_STEPS} + \text{PLANT_STEPS} + \text{PUSH_STEPS}}$$

Per-leg command law

Let b_l be baseline abduction offset (front/mid/rear specific), and a_l be stride amplitude (front/mid/rear specific). Define signed stride direction:

$$\sigma_l(\varphi_l) = \begin{cases} +1, & \text{swing} \\ -1, & \text{push} \\ 0, & \text{lift/plant transition} \end{cases}$$

Abduction command:

$$q_l^{\text{abd}} = b_l + s_{\text{stride}} \cdot a_l \cdot \sigma_l(\varphi_l)$$

Flexion command:

$$q_l^{\text{flx}} = \begin{cases} q_{\text{lift}}, & \text{lift} \\ q_{\text{lift}}, & \text{swing} \\ q_{\text{plant}}, & \text{plant transition} \\ q_{\text{push}}, & \text{stance push} \end{cases}$$

where:

$$q_{\text{lift}} = q_{\text{lift}}^0 \cdot s_{\text{height}}, \quad q_{\text{push}} = q_{\text{push}}^0, \quad q_{\text{plant}} = q_{\text{start}}^0$$

Roll/pitch coupling distribution across legs

Reflex corrections are coupled by side and body-axis:

$$\Delta_{\text{roll}} = K_{\phi}\phi + K_{\dot{\phi}}\dot{\phi} \tag{1}$$

$$\Delta_{\text{pitch}} = K_{\theta}\theta + K_{\dot{\theta}}\dot{\theta} \tag{2}$$

For each leg l , define side sign $s_l \in \{+1, -1\}$ and front-rear sign $r_l \in \{+1, 0, -1\}$:

- $s_l = +1$ for left legs (FL, ML, RL), $s_l = -1$ for right legs (FR, MR, RR)
- $r_l = -1$ front (FL, FR), $r_l = 0$ middle (ML, MR), $r_l = +1$ rear (RL, RR)

Then leg flex bias is:

$$\Delta q_l^{\text{flx}} = s_l \Delta_{\text{roll}} + r_l \Delta_{\text{pitch}}$$

Final flex command before pulse conversion:

$$\hat{q}_l^{\text{flx}} = q_l^{\text{flx}} + \Delta q_l^{\text{flx}}$$

followed by hard clamping and slew-rate limiting.

Why this coupling is implementable now

This coupling model is directly compatible with the existing v2 structure:

- Current tripod functions already encode anti-phase A/B behavior
- Existing front/mid/rear amplitudes map to a_l
- Existing start/lift/plant/push flex constants map to q^0
- Existing clamp and interpolation functions enforce safety and smoothness

6 Core Parameterization

Runtime gait parameter vector

Define the control parameter vector:

$$\mathbf{p} = \left[q_{\text{start}}^0 \quad q_{\text{push}}^0 \quad q_{\text{lift}}^0 \quad q_{\text{plant}}^0 \quad a_F \quad a_M \quad a_R \quad \omega \quad s_{\text{stride}} \quad s_{\text{height}} \right]^\top$$

where (a_F, a_M, a_R) are front/middle/rear abduction amplitudes and ω is gait angular frequency.

State, action, and observation

At fast tick k (period Δt), define:

$$\mathbf{x}_k = \left[\phi_k \quad \theta_k \quad \dot{\phi}_k \quad \dot{\theta}_k \quad \varphi_k \right]^\top, \quad \mathbf{u}_k = \left[\Delta \mathbf{p}_k \right]$$

with observation $\mathbf{o}_k = \mathbf{x}_k$ in the minimal onboard setup.

Online adaptation parameter subset

For stability-first adaptation on hardware:

$$\boldsymbol{\theta} = \left[K_\phi \quad K_{\dot{\phi}} \quad K_\theta \quad K_{\dot{\theta}} \quad s_{\text{stride}} \quad s_{\text{height}} \quad s_{\text{freq}} \right]^\top$$

This is the vector updated online (SPSA/bandit/PPO output), while core safety bounds remain fixed.

7 Control and Stability Formulation

Phase oscillator and hybrid gait dynamics

Global gait phase:

$$\varphi_{k+1} = (\varphi_k + \omega_k \Delta t) \bmod 1 \tag{3}$$

Per-leg local phase:

$$\varphi_{l,k} = (\varphi_k + \delta_l) \bmod 1, \quad \delta_l \in \{0, 0.5\} \tag{4}$$

Piecewise gait mode map:

$$m(\varphi_{l,k}) \in \{\text{lift}, \text{swing}, \text{plant}, \text{push}\}$$

with switching boundaries (τ_1, τ_2, τ_3) obtained from step-count ratios.

Per-leg command generator

For each leg l :

$$q_{l,k}^{\text{abd}} = b_l + s_{\text{stride},k} a_{g(l)} \sigma(m(\varphi_{l,k})) \quad (5)$$

$$q_{l,k}^{\text{flx,base}} = \begin{cases} q_{\text{lift}}^0 s_{\text{height},k}, & m = \text{lift/swing} \\ q_{\text{plant}}^0, & m = \text{plant} \\ q_{\text{push}}^0, & m = \text{push} \end{cases} \quad (6)$$

where $g(l) \in \{F, M, R\}$ maps each leg to front/middle/rear class.

Reflex-coupled stabilization law

$$\Delta_{\text{roll},k} = K_{\phi} \phi_k + K_{\dot{\phi}} \dot{\phi}_k \quad (7)$$

$$\Delta_{\text{pitch},k} = K_{\theta} \theta_k + K_{\dot{\theta}} \dot{\theta}_k \quad (8)$$

Leg-specific flex coupling:

$$\Delta q_{l,k}^{\text{flx}} = s_l \Delta_{\text{roll},k} + r_l \Delta_{\text{pitch},k} \quad (9)$$

with side sign $s_l \in \{+1, -1\}$ and front-rear sign $r_l \in \{-1, 0, +1\}$.

Final command before pulse conversion:

$$\hat{q}_{l,k}^{\text{flx}} = q_{l,k}^{\text{flx,base}} + \Delta q_{l,k}^{\text{flx}} \quad (10)$$

Safety projection and slew constraints

Pulse command for servo channel $c(l, j)$:

$$u_{c,k}^{\mu s} = \text{clip}\left(\mu_c + d_c \hat{q}_{l,j,k}, u_c^{\min}, u_c^{\max}\right) \quad (11)$$

where μ_c is servo center and $d_c \in \{-1, +1\}$ is direction.

Slew-limited command:

$$u_{c,k}^{\mu s} \leftarrow \text{clip}\left(u_{c,k}^{\mu s}, u_{c,k-1}^{\mu s} - \Delta_c^{\max}, u_{c,k-1}^{\mu s} + \Delta_c^{\max}\right) \quad (12)$$

Instability metric and mode switching

Instantaneous score:

$$S_k = w_1 |\phi_k| + w_2 |\theta_k| + w_3 |\dot{\phi}_k| + w_4 |\dot{\theta}_k| \quad (13)$$

Windowed RMS:

$$S_{\text{rms}}(n) = \sqrt{\frac{1}{N} \sum_{i=n-N+1}^n S_i^2} \quad (14)$$

Hybrid mode:

$$\text{mode} = \begin{cases} \text{cautious}, & S_{\text{rms}} > \gamma_H \text{ for } M_H \text{ cycles} \\ \text{nominal}, & S_{\text{rms}} < \gamma_L \text{ for } M_L \text{ cycles} \end{cases}$$

with $\gamma_H > \gamma_L$ (hysteresis).

Online adaptation objective and update

Per-window cost:

$$J = \alpha_1 \text{RMS}(\phi) + \alpha_2 \text{RMS}(\theta) + \alpha_3 \text{mean}(|\dot{\phi}| + |\dot{\theta}|) + \alpha_4 \mathbb{I}_{\text{fall}} + \alpha_5 E_{\text{cmd}} \quad (15)$$

where

$$E_{\text{cmd}} = \frac{1}{NT} \sum_{k=1}^N \sum_{c=1}^{12} |u_{c,k}^{\mu s} - u_{c,k-1}^{\mu s}|$$

is a command-variation proxy for effort.

SPSA update:

$$\boldsymbol{\theta}^+ = \boldsymbol{\theta}_n + c_n \boldsymbol{\Delta}_n, \quad \boldsymbol{\theta}^- = \boldsymbol{\theta}_n - c_n \boldsymbol{\Delta}_n \quad (16)$$

$$\hat{\mathbf{g}}_n = \frac{J(\boldsymbol{\theta}^+) - J(\boldsymbol{\theta}^-)}{2c_n} \odot \boldsymbol{\Delta}_n^{-1} \quad (17)$$

$$\boldsymbol{\theta}_{n+1} = \Pi_{[\ell, \mathbf{u}]}(\boldsymbol{\theta}_n - a_n \hat{\mathbf{g}}_n) \quad (18)$$

where $\Pi_{[\ell, \mathbf{u}]}$ is box projection to safety bounds.

PPO objective for simulation policy

For policy $\pi_\psi(\Delta\boldsymbol{\theta} \mid \mathbf{o})$, use clipped objective:

$$\mathcal{L}^{\text{CLIP}}(\psi) = \mathbb{E}_t \left[\min \left(r_t(\psi) \hat{A}_t, \text{clip}(r_t(\psi), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (19)$$

with

$$r_t(\psi) = \frac{\pi_\psi(a_t \mid o_t)}{\pi_{\psi_{\text{old}}}(a_t \mid o_t)}$$

The whiteboard intent is to tune architecture parameters from IMU-defined reward only. Define IMU signal vector:

$$\mathbf{z}_t = \begin{bmatrix} \phi_t & \theta_t & \dot{\phi}_t & \dot{\theta}_t & \ddot{\phi}_t & \ddot{\theta}_t \end{bmatrix}^\top$$

with $\ddot{\phi}_t, \ddot{\theta}_t$ estimated by finite differences on gyro rates.

Primary IMU reward:

$$r_t^{\text{IMU}} = \exp\left(-\mathbf{z}_t^\top \mathbf{W} \mathbf{z}_t\right) - \beta_f \mathbb{I}_{\text{fall},t}$$

where $\mathbf{W} \succeq 0$ is diagonal in implementation.

Equivalent weighted linearized form (for diagnostics and ablation):

$$r_t^{\text{lin}} = -\beta_\phi |\phi_t| - \beta_\theta |\theta_t| - \beta_{\dot{\phi}} |\dot{\phi}_t| - \beta_{\dot{\theta}} |\dot{\theta}_t| - \beta_{\ddot{\phi}} |\ddot{\phi}_t| - \beta_{\ddot{\theta}} |\ddot{\theta}_t| - \beta_f \mathbb{I}_{\text{fall},t}.$$

8 Completion-Oriented Execution (No Calendar Assumption)

Completion Gate 0 (Mandatory): IMU signal availability on robot

- Bring up IMU hardware and firmware path so the controller receives live measurements each fast tick
- Validate timestamped streams for:

$$\{a_x, a_y, a_z, g_x, g_y, g_z\}_t$$

with bounded drop rate and bounded jitter

- Estimate:

$$(\phi_t, \theta_t, \dot{\phi}_t, \dot{\theta}_t, \ddot{\phi}_t, \ddot{\theta}_t)$$

and verify physical plausibility under known motions

Gate criterion 0 (must pass before any training):

dropout_rate < 1%, rms_timestamp_jitter < 0.1 Δt , bias_drift bounded in static test

Completion Block A: Architecture lock-in

- Implement (φ, φ_l) oscillator formalism and unified per-leg law in `hexapod_walking_v3`
- Replace hardcoded phase functions with table-driven (τ_1, τ_2, τ_3) switching
- Define architecture interfaces explicitly:

$$\mathbf{o}_t \xrightarrow{\pi_\psi} \Delta \boldsymbol{\theta}_t, \quad (\boldsymbol{\theta}_t, \varphi_t) \xrightarrow{\text{gait map}} \mathbf{q}_t, \quad \mathbf{q}_t \xrightarrow{\text{servo map}} \mathbf{u}_t^{\mu s}$$

- Instrument full telemetry tuple:

$$\{\varphi, \varphi_l, m_l, q_l, \hat{q}_l, u_c^{\mu_s}, S_k, \text{mode}\}$$

- Deliver parameter-identification script for $\mu_c, d_c, u_c^{\min}, u_c^{\max}, \Delta_c^{\max}$

Completion criterion A:

$$\left| \frac{T_{v3} - T_{v2}}{T_{v2}} \right| < 0.05, \quad N_{\text{clamp},v3} \leq N_{\text{clamp},v2}$$

under matched test conditions.

Completion Block B: IMU reward and stabilization closure

- Integrate IMU estimator at fast tick and compute $(\phi, \theta, \dot{\phi}, \dot{\theta})$
- Implement coupled bias law $\Delta q_l^{\text{fix}} = s_l \Delta_{\text{roll}} + r_l \Delta_{\text{pitch}}$
- Add hysteretic mode switch $(\gamma_H, \gamma_L, M_H, M_L)$ with bounded reparameterization:

$$s_{\text{stride}} \downarrow, \quad s_{\text{height}} \uparrow, \quad \omega \downarrow$$

- Run disturbance matrix: frontal push, lateral push, incline start, uneven contact start

Completion criterion B:

$$\Pr(\text{fall})_{\text{reflex}} \leq 0.7 \Pr(\text{fall})_{v2}, \quad \bar{S}_{\text{rms},\text{reflex}} < \bar{S}_{\text{rms},v2}$$

with identical disturbance protocol.

Training lock: Completion Blocks C and D are not allowed unless Gate 0 and Block B are passed.

Completion Block C: Parameter tuning engine

- Fix tunable architecture parameter set:

$$\boldsymbol{\theta} = [K_\phi, K_{\dot{\phi}}, K_\theta, K_{\dot{\theta}}, s_{\text{stride}}, s_{\text{height}}, s_{\text{freq}}]$$

- Use projected SPSA as deterministic pre-training/tuning baseline
- Add EEPROM persistence for best $\boldsymbol{\theta}^*$ by cost J
- Produce ablation: reflex-only vs reflex+SPSA over identical terrain trials

Completion criterion C:

$$\Delta J_{\text{SPSA}} = J(\boldsymbol{\theta}^*) - J(\boldsymbol{\theta}_0) < 0$$

with confidence interval excluding 0 across repeated trials.

Completion Block D: PPO parameter tuner and transfer

- Train PPO in `sim/` with randomized contact/friction/sensor-noise/delay while keeping architecture fixed
- Restrict policy output to bounded parameter updates:

$$\Delta \boldsymbol{\theta}_t = \pi_\psi(\mathbf{o}_t), \quad \boldsymbol{\theta}_{t+1} = \Pi_{[\ell, u]}(\boldsymbol{\theta}_t + \Delta \boldsymbol{\theta}_t)$$

- Validate transfer on three terrains with fixed evaluation protocol
- Final deliverables: architecture equations, PPO tuning equations, implementation logs, and demo script

Completion criterion D: PPO-assisted controller outperforms reflex+SPSA baseline on IMU reward functional:

$$\mathcal{R}_{\text{IMU}} = \frac{1}{T} \sum_{t=1}^T r_t^{\text{IMU}}$$

while preserving all safety constraints.

9 Safety Envelope (Non-Negotiable)

1. **Hard pulse bounds:** never bypass `MIN_US/MAX_US`
2. **Per-parameter bounds:** each tunable variable has fixed min/max
3. **Slew-rate limits:** no abrupt parameter jumps between cycles
4. **Fall detection:** if $|\phi|$ or $|\theta|$ exceeds critical angle, stop gait and enter safe mode
5. **Recovery policy:** controlled restart sequence only

Bounded parameter box:

$$\ell_i \leq \theta_i \leq u_i, \quad |\theta_{i,k+1} - \theta_{i,k}| \leq \Delta_{\theta_i}^{\max}$$

for every adaptive parameter θ_i .

10 Repository Integration Plan

Path	Planned role
firmware/hexapod_walking_v2/	Reference controller for regression checks
firmware/hexapod_walking_v2/	Run-time-parameterized gait, reflexes, adaptation hooks
sim/	PPO training, randomization, ablations, policy export
host/	Experiment runner, telemetry parser, parameter management
docs/	Benchmarks, test protocols, safety limits, milestone evidence

11 Practical To-Do Checklist (How To Proceed)

Step 1: Bring up IMU pipeline first (hard prerequisite)

- Wire and initialize IMU in firmware; stream raw accel/gyro to logs
- Implement calibration routine (bias + axis sign/scale sanity checks)
- Add health monitors: sample frequency, dropout count, timestamp jitter
- Compute filtered orientation and rates:

$$(\phi, \theta, \dot{\phi}, \dot{\theta})$$

and second derivatives via finite differences

Artifact: IMU health report with pass/fail on rate, jitter, and drift.

Step 2: Freeze baseline and create regression harness

- Duplicate current controller as immutable baseline:

$$\text{hexapod_walking_v2} \rightarrow \text{baseline_frozen}$$

- Create one command/script that runs fixed-duration trials and exports logs
- Log at minimum: phase, fall flag, clamp count, and IMU-derived state

Artifact: baseline log bundle + one reproducible run command.

Step 3: Implement architecture skeleton in v3

- Create `hexapod_walking_v3` with explicit pipeline:

$$\mathbf{o}_t \rightarrow \Delta\boldsymbol{\theta}_t \rightarrow \mathbf{q}_t \rightarrow \mathbf{u}_t^{\mu s}$$

- Move all gait constants into a runtime parameter struct/vector
- Keep outputs numerically close to baseline before adding adaptation

Artifact: v3 firmware that reproduces v2 behavior with runtime parameters.

Step 4: Define IMU reward and verify observability

- Compute IMU state vector:

$$\mathbf{z}_t = [\phi_t, \theta_t, \dot{\phi}_t, \dot{\theta}_t, \ddot{\phi}_t, \ddot{\theta}_t]^\top$$

- Implement both reward forms:

$$r_t^{\text{IMU}} = \exp(-\mathbf{z}_t^\top \mathbf{W} \mathbf{z}_t) - \beta_f \mathbb{I}_{\text{fall},t}, \quad r_t^{\text{lin}}$$

- Verify reward traces correlate with visible stability

Artifact: reward debug plot and IMU-reward consistency notes.

Step 5: Implement leg-coupled reflex controller

- Implement per-leg flex bias:

$$\Delta q_l^{\text{flex}} = s_l \Delta_{\text{roll}} + r_l \Delta_{\text{pitch}}$$

- Add hysteretic cautious mode and bounded reparameterization
- Validate anti-phase tripod coupling remains intact

Artifact: disturbance-test table (baseline vs reflex).

Step 6: Enforce hard safety constraints everywhere

- Pulse bounds, parameter bounds, and slew bounds must all be active:

$$\ell_i \leq \theta_i \leq u_i, \quad |\theta_{i,k+1} - \theta_{i,k}| \leq \Delta_{\theta_i}^{\text{max}}$$

- Add fail-safe behavior for critical tilt/fall events
- Record every safety intervention in logs

Artifact: safety validation checklist with pass/fail outcomes.

Step 7: Run projected SPSA as a tuning baseline

- Optimize:

$$\boldsymbol{\theta} = [K_\phi, K_{\dot{\phi}}, K_\theta, K_{\dot{\theta}}, s_{\text{stride}}, s_{\text{height}}, s_{\text{freq}}]$$

- Use projected update:

$$\boldsymbol{\theta}_{n+1} = \Pi_{[\ell, u]}(\boldsymbol{\theta}_n - a_n \hat{\mathbf{g}}_n)$$

- Persist best $\boldsymbol{\theta}^*$ to EEPROM

Artifact: SPSA run history + best-parameter snapshot.

Step 8: Train PPO as architecture-parameter tuner

- Keep gait architecture fixed; policy outputs only $\Delta\boldsymbol{\theta}$
- Train with randomized simulator physics/sensor delay/noise
- Use clipped PPO objective and IMU reward
- Do not train/deploy if IMU health report fails Gate 0 thresholds

Artifact: trained checkpoint + validation curves.

Step 9: Transfer PPO policy to hardware safely

- Apply policy output through projection before firmware update:

$$\boldsymbol{\theta}_{t+1} = \Pi_{[\ell, u]}(\boldsymbol{\theta}_t + \Delta\boldsymbol{\theta}_t)$$

- Execute fixed evaluation protocol on multiple terrains
- Compare baseline, reflex-only, SPSA, and PPO controllers

Artifact: final comparison report and demo script.

Step 10: Lock final submission package

- Freeze code commit/tag for final architecture and policy
- Save reproducibility package: logs, params, scripts, plots, and PDF
- Ensure a single “run-eval” command reproduces headline metrics

Artifact: final reproducible release bundle.

12 Definition of Success (Quantitative)

Let baseline be v2 and candidate be the architecture+PPO controller. Success requires:

$$\frac{J_{\text{candidate}} - J_{\text{v2}}}{J_{\text{v2}}} \leq -0.25 \quad (20)$$

$$\Pr(\text{fall} \mid \text{disturbance set})_{\text{candidate}} \leq 0.7 \Pr(\text{fall} \mid \text{disturbance set})_{\text{v2}} \quad (21)$$

$$\mathcal{R}_{\text{IMU,candidate}} > \mathcal{R}_{\text{IMU,v2}} \quad (22)$$

plus full reproducibility of logs, parameter files, and run scripts.